

Despliegue automático de una aplicación escalable en OpenStack mediante Terraform.

Grupo21 - Jorge Ortega Rodríguez y Jorge Suárez Souto

Índice

1. Introducción.....	1
2. Desarrollo	1
2.1. Topología del escenario y requerimientos funcionales	1
2.2. Configuración del escenario.....	2
2.2.1. Configuración de las redes y subredes	2
2.2.2. Configuración del router y encaminamiento	3
2.2.3. Configuración de las instancias (web, administración y base de datos).....	3
2.2.4. Configuración del balanceador de carga.....	5
2.2.5. Configuración de reglas y políticas de firewall	5
2.2.6. Configuración de los grupos de seguridad	6
3. Pruebas	6
3.1. Funcionamiento del balanceador de carga	7
3.2. Funcionamiento de los servidores web	7
3.3. Funcionamiento de la base de datos	8
3.4. Acceso a la BBDD desde servidores web	8
3.5. Acceso al servidor de administración desde el exterior	9
3.6. Acceso a los servidores web desde el admin	10
3.7. Acceso desde el interior a cualquier destino	11

1. Introducción

En este proyecto se implementa un entorno en OpenStack orquestado mediante Terraform, siguiendo el enfoque de Infraestructura como Código (IaC), donde la infraestructura se define de forma declarativa.

El objetivo es desplegar una aplicación web escalable compuesta por varios servidores web, un componente de base de datos aislado, un balanceador de carga que distribuya el tráfico entrante y un control de seguridad que restrinja el acceso desde el exterior a lo estrictamente necesario. Todo el escenario se crea y configura automáticamente, incluyendo la inicialización de las máquinas virtuales mediante cloud-init.

2. Desarrollo

2.1. Topología del escenario y requerimientos funcionales

Topología lógica

El escenario se construye sobre una red externa proporcionada por OpenStack (ExtNet) y dos redes internas:

- ExtNet (red externa): proporciona direcciones IP públicas (IPs flotantes) que permiten exponer elementos concretos del escenario.
- Net1 (red de servicio / acceso): red donde se ubica el punto de entrada del servicio y donde se conecta el router hacia ExtNet.
- Net2 (red de datos / back-end): red privada destinada a tráfico interno, especialmente para el servidor de base de datos.

La conectividad se diseña con un principio claro:

- La salida/entrada hacia Internet se concentra en Net1.
- La red Net2 no se enruta hacia ExtNet, de manera que el servidor de base de datos queda naturalmente aislado.

Requerimientos funcionales implementados

1. **Inicialización automática con cloud-init:**
 - Web: instalación y arranque de Apache, despliegue de página inicial.
 - Administración: cambio del puerto SSH al 2025 y ajuste de parámetros de acceso.
 - Base de datos: disponibilidad de MariaDB y arranque del servicio.
2. **Balanceo de carga:**
 - Publicación del servicio HTTP en Net1 y reparto entre s1/s2/s3 con algoritmo ROUND_ROBIN.
3. **Seguridad y control de accesos:**

- Acceso externo permitido únicamente a:
 - HTTP/80 hacia el balanceador.
 - SSH/2025 hacia el servidor admin.
 - Acceso interno (desde la red de servicio) con salida libre a destinos externos a través del router, sujeto a la política configurada.
4. **Aislamiento de base de datos:**
- El servidor de base de datos se conecta exclusivamente a Net2, y Net2 no dispone de ruta directa a ExtNet.

2.2. Configuración del escenario

2.2.1. Configuración de las redes y subredes

Red Net1 (servicio / acceso)

Se crea una red interna destinada a albergar el tráfico de entrada del servicio web y la conectividad hacia el exterior. A Net1 se le asocia una subred con:

- Direccionamiento IPv4 dentro del rango 10.0.1.0/24.
- Configuración de DNS (servidores públicos) para permitir resolución de nombres desde las instancias que utilicen esta red como salida.

Esta red cumple dos funciones:

1. Alojar el **VIP del balanceador**, que es el punto de entrada lógico del servicio web.
2. Servir como red conectada al router que enlaza con ExtNet (salida a Internet).

Red Net2 (datos / back-end)

Se crea una red interna separada para tráfico de datos y comunicaciones internas. A Net2 se le asocia una subred:

- Direccionamiento IPv4 dentro del rango 10.0.2.0/24.

Esta red está pensada para:

- Interconectar los servidores web con el servidor de base de datos.
- Mantener el servidor de base de datos en un segmento no publicado.

Puerto de gateway en Net1

Además de crear la red, se define explícitamente un puerto en Net1 con IP fija 10.0.1.1, que actúa como punto de enlace del encaminamiento (interfaz del router) y, más importante, como punto de aplicación de las políticas de firewall. Esta decisión permite:

- Controlar de forma precisa el “punto de paso” del tráfico norte-sur.
- Asociar FWaaS a una interfaz concreta que representa la puerta de salida/entrada.

2.2.2. Configuración del router y encaminamiento

Para proporcionar conectividad con el exterior desde Net1, se despliega un router conectado a:

- ExtNet (red externa), como interfaz “uplink”.
- Net1, mediante la interfaz asociada al puerto fijo 10.0.1.1.

La decisión arquitectónica clave es que Net2 no se conecta al router, lo que implica:

- Net1 dispone de salida/entrada hacia ExtNet.
- Net2 queda sin ruta directa al exterior, reforzando el aislamiento del servidor de base de datos.

Este modelo es coherente con un enfoque de seguridad por segmentación: solo se publica lo estrictamente necesario y el componente de datos permanece en una red interna.

2.2.3. Configuración de las instancias (web, administración y base de datos)

Parámetros comunes

Las instancias se despliegan a partir de una imagen Ubuntu y un “flavor” de tamaño reducido adecuado para el escenario. Todas las instancias se asocian a un grupo de seguridad común y se configuran con claves SSH específicas.

Además, se introducen dependencias explícitas en el despliegue para asegurar que redes y subredes estén listas antes de crear máquinas virtuales, evitando fallos por orden de creación.

Servidores web: s1, s2 y s3

Se despliegan tres instancias web con dos interfaces de red cada una:

- Interfaz en Net1: destinada al tráfico servido al balanceador y, por extensión, a los clientes.
- Interfaz en Net2: destinada a comunicación interna con la base de datos (capa back-end).

Configuración automática en primer arranque (cloud-init):

- Actualización de repositorios del sistema.
- Instalación de Apache2 como servidor web.
- Instalación de herramientas auxiliares.
- Creación automática de una página index.html en el directorio servido por Apache.
- Habilitación del servicio Apache para que arranque en reinicios.
- Arranque inmediato del servicio.

De esta forma, cada servidor web queda operativo sin intervención manual: al finalizar el primer arranque, ya responde por HTTP en el puerto 80.

Servidor de administración: admin

Se despliega un servidor de administración con conectividad a:

- Net1, para poder gestionar el entorno y ser accesible por la vía controlada desde el exterior.
- Net2, para poder alcanzar también el plano de datos y administrar el servidor de base de datos si es necesario.

Configuración automática (cloud-init) orientada a acceso administrativo:

- Actualización de paquetes del sistema.
- Modificación de la configuración del servicio SSH para:
 - Permitir acceso como root.
 - Permitir autenticación por contraseña.
 - Cambiar el puerto de escucha de SSH a 2025.
- Reinicio del servicio SSH para aplicar cambios.

Posteriormente, se asocia una IP flotante a este servidor, de modo que sea el único punto de entrada SSH desde el exterior. El filtrado de acceso (puerto 2025) lo realiza el firewall del escenario.

Servidor de base de datos

Se despliega una instancia de base de datos conectada únicamente a Net2, reforzando el aislamiento de datos.

En cuanto a la imagen utilizada, se parte de una imagen tipo “snapshot” destinada a base de datos, estrategia habitual cuando:

- Se quiere asegurar que el servicio principal (MariaDB) ya está disponible sin depender de descargas externas desde una red aislada.
- Se desea mantener el servidor de base de datos en una red sin salida a Internet.

Configuración automática (cloud-init) orientada al servicio:

- Instalación de MariaDB.
- Habilitación del servicio para reinicios.
- Arranque inmediato del motor de base de datos.

Resultado: el servidor de base de datos queda operativo y accesible desde la red interna Net2 por las instancias que dispongan de interfaz en esa red.

2.2.4. Configuración del balanceador de carga

El balanceador se despliega con el servicio nativo de OpenStack (Octavia) y se compone conceptualmente de cuatro piezas:

1. **Load Balancer (LB)**

Se crea un balanceador ubicado en la subred de Net1. Ese balanceador es la dirección interna a la que se enviará el tráfico entrante antes de distribuirlo hacia los servidores web.

2. **Listener HTTP**

Se configura un listener para aceptar tráfico HTTP en el puerto 80, estableciendo el punto de recepción del tráfico del servicio.

3. **Pool**

Se define un pool asociado al listener, que agrupa los servidores web y define el método de reparto. Se utiliza ROUND_ROBIN, distribuyendo peticiones de forma cíclica y equitativa entre s1, s2 y s3.

4. **Members**

Se añaden como miembros del pool los tres servidores web utilizando sus direcciones IP internas en Net1 y el puerto 80 como destino.

Finalmente, para publicar el servicio hacia el exterior, se reserva una IP flotante y se asocia al puerto del VIP del balanceador. Así, desde Internet se accede a la IP pública del balanceador, y éste distribuye a los servidores web internos.

2.2.5. Configuración de reglas y políticas de firewall

El control de tráfico se implementa mediante FWaaS con una metodología estándar:

1. **Reglas individuales:** definen permisos concretos para tipos de tráfico.
2. **Políticas:** agrupan reglas para tráfico entrante (ingress) y saliente (egress).
3. **Grupo de firewall:** aplica esas políticas a un puerto específico del escenario (en este caso, el puerto gateway de Net1).

Reglas definidas

Se implementan tres reglas principales:

1. **Acceso SSH al servidor de administración**

- Protocolo: TCP
 - Puerto destino: 22
 - Origen: cualquier IP externa (0.0.0.0/0)
 - Destino: IP interna del servidor admin en Net1
- Objetivo: permitir administración remota únicamente por el puerto establecido.

2. **Acceso HTTP al balanceador de carga**

- Protocolo: TCP
- Puerto destino: 80

- Origen: cualquier IP externa (0.0.0.0/0)
 - Destino: VIP interno del balanceador
- Objetivo: publicar el servicio web escalable hacia el exterior.

3. Tráfico interno con permiso amplio

- Protocolo: cualquiera
 - Origen: red Net1 (10.0.1.0/24)
- Objetivo: permitir tráfico originado desde dentro del escenario (Net1) con libertad de protocolo, facilitando conectividad interna y salida.

Políticas de firewall

- **Política de entrada (ingress):** agrupa la regla HTTP al balanceador, la regla SSH al admin y la regla interna.
- **Política de salida (egress):** permite tráfico originado desde la red interna Net1 (regla interna).

Aplicación del firewall

El firewall se aplica al puerto gateway de Net1 (10.0.1.1). Esto tiene una ventaja metodológica clara: todo el tráfico norte-sur pasa por ese punto, por lo que la política es efectiva y centralizada.

2.2.6. Configuración de los grupos de seguridad

Los grupos de seguridad complementan el firewall y controlan el tráfico a nivel de instancia (tráfico este-oeste y parte del norte-sur, según el caso). En este escenario se define un grupo de seguridad común denominado open, asociado a todas las instancias.

Reglas implementadas

El grupo se configura para permitir:

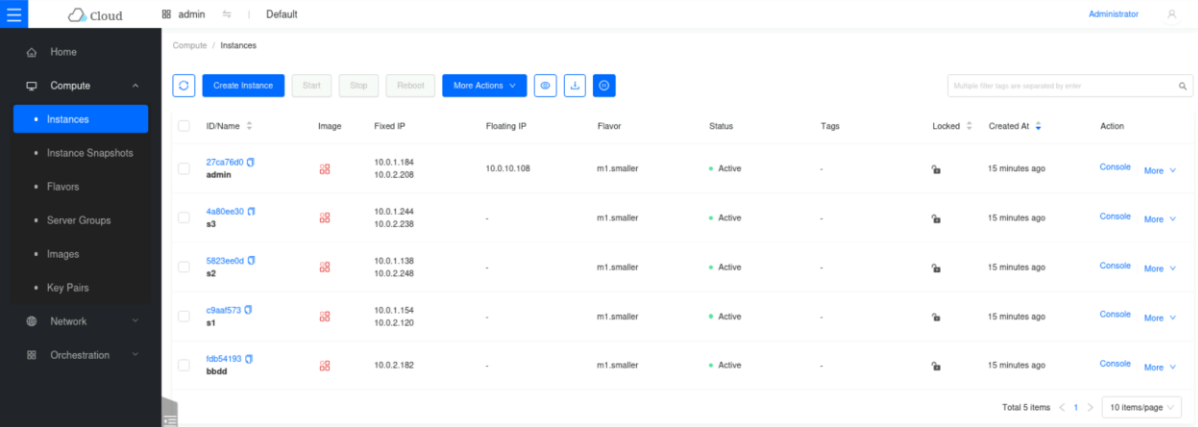
- Tráfico TCP entrante de forma amplia.
- Tráfico TCP saliente de forma amplia.
- Tráfico ICMP entrante y saliente (útil para pruebas de conectividad mediante ping).
- Una regla específica para permitir SSH en el puerto 22 (orientada a pruebas o administración alternativa).

3. Pruebas

En esta sección se recogen las verificaciones realizadas para demostrar que el escenario desplegado funciona correctamente y cumple los requisitos de conectividad, aislamiento y publicación de servicios. En la siguiente imagen se ven las direcciones de todos los recursos.

3.1. Funcionamiento del balanceador de carga

Para verificar el correcto funcionamiento del balanceador de carga, se realizaron peticiones HTTP a la dirección IP flotante asociada al balanceador, que se muestra en la Figura 1.



ID/Name	Image	Fixed IP	Floating IP	Flavor	Status	Tags	Locked	Created At	Action
27ca7640 admin		10.0.1.184 10.0.2.208	10.0.10.108	m1.small	Active	-		15 minutes ago	Console More
4a80ee30 s3		10.0.1.244 10.0.2.238	-	m1.small	Active	-		15 minutes ago	Console More
5823ee0d s2		10.0.1.138 10.0.2.248	-	m1.small	Active	-		15 minutes ago	Console More
c9ad573 s1		10.0.1.154 10.0.2.120	-	m1.small	Active	-		15 minutes ago	Console More
4b64193 bddd		10.0.2.182	-	m1.small	Active	-		15 minutes ago	Console More

Figura 1: Direcciones de los recursos de la arquitectura.

A continuación, se ejecutaron varias peticiones secuenciales a dicha IP. Como parte de la configuración inicial (cloud-init), se personalizó el contenido del fichero index.html en cada servidor web para poder identificar visualmente qué backend estaba respondiendo. El resultado esperado es que, al usar un algoritmo de reparto tipo *round-robin*, las respuestas vayan alternando entre los tres servidores.

Las evidencias muestran que, al repetir accesos consecutivos a la IP flotante del balanceador, el contenido de la página cambia, lo que confirma que el balanceador distribuye correctamente el tráfico entre s1, s2 y s3.

```
root@terraform:~/shared# for i in {1..10}; do curl http://10.0.10.113; done
Mensaje VM1
Mensaje VM3
Mensaje VM2
Mensaje VM1
Mensaje VM3
Mensaje VM2
Mensaje VM1
Mensaje VM3
Mensaje VM2
Mensaje VM1
root@terraform:~/shared#
```

Figura 2: Demostración del balanceador de carga

3.2. Funcionamiento de los servidores web

Con el objetivo de verificar que cada servidor web está operativo y en estado saludable, se accedió a cada instancia y se comprobó el estado del servicio Apache mediante el comando `systemctl status apache2`

En la Figura 3 se muestra el estado correcto del servicio en el servidor s1, evidenciando que Apache se encuentra activo y funcionando adecuadamente.

```
root@s1:~# systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-01-17 18:33:04 UTC; 2min 42s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2667 (apache2)
     Tasks: 55 (limit: 527)
    Memory: 5.2M
   CGroup: /system.slice/apache2.service
           └─2667 /usr/sbin/apache2 -k start
             └─2670 /usr/sbin/apache2 -k start
               └─2671 /usr/sbin/apache2 -k start

Jan 17 18:33:04 s1 systemd[1]: Starting The Apache HTTP Server...
Jan 17 18:33:04 s1 apachectl[2666]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, us
Jan 17 18:33:04 s1 systemd[1]: Started The Apache HTTP Server.
lines 1-15/15 (END)
```

Figura 3: Estado del servidor 1

Para simplificar el documento no se adjuntan el resto de capturas pertenecientes al resto de servidores.

3.3. Funcionamiento de la base de datos

Se verificó también el correcto funcionamiento del gestor de base de datos instalado (MariaDB) en el servidor de datos. Para ello, se comprobó el estado del servicio con `systemctl status mariadb`

En la Figura 4 se observa el estado saludable del servicio, confirmando que la base de datos se encuentra levantada y operativa.

```
root@bbdd:~# systemctl status mariadb
● mariadb.service - MariaDB 10.6.22 database server
   Loaded: loaded (/lib/systemd/system/mariadb.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-01-17 18:29:35 UTC; 7min ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Main PID: 763 (mariadb)
   Status: "Taking your SQL requests now..."
     Tasks: 8 (limit: 3379)
    Memory: 70.2M
       CPU: 1.118s
   CGroup: /system.slice/mariadb.service
           └─763 /usr/sbin/mariadb

Jan 17 18:29:20 bbdd mariadb[763]: 2026-01-17 18:29:20 0 [Note] Plugin 'FEEDBACK' is disabled.
Jan 17 18:29:21 bbdd mariadb[763]: 2026-01-17 18:29:21 0 [Note] InnoDB: Buffer pool(s) load completed at 260117 18:29:21
Jan 17 18:29:28 bbdd mariadb[763]: 2026-01-17 18:29:28 0 [Warning] You need to use --log-bin to make --expire-logs-days or --b
Jan 17 18:29:28 bbdd mariadb[763]: 2026-01-17 18:29:28 0 [Note] Server socket created on IP: '0.0.0.0'.
Jan 17 18:29:35 bbdd mariadb[763]: 2026-01-17 18:29:35 0 [Note] /usr/sbin/mariadb: ready for connections.
Jan 17 18:29:35 bbdd mariadb[763]: Version: '10.6.22-MariaDB-0ubuntu0.22.04.1' socket: '/run/mysqld/mysqld.sock' port: 3306
Jan 17 18:29:35 bbdd systemd[1]: Started MariaDB 10.6.22 database server.
Jan 17 18:29:35 bbdd /etc/mysql/debian-start[924]: Upgrading MySQL tables if necessary.
Jan 17 18:29:39 bbdd /etc/mysql/debian-start[939]: Checking for insecure root accounts.
Jan 17 18:29:40 bbdd /etc/mysql/debian-start[946]: Triggering myisam-recover for all MyISAM tables and aria-recover for all Ari
lines 1-23/23 (END)
```

Figura 4: Estado del gestor de base de datos

3.4. Acceso a la BBDD desde servidores web

En este apartado se comprueba que los servidores web tienen conectividad con el servidor de base de datos a través de la red interna correspondiente.

En la Figura 5 se observa el resultado de ejecutar ping y el intento de conexión ssh, confirmando que desde los servidores web existe alcance hacia el servidor de base de datos.

```
root@s2:~# ping 10.0.2.182
PING 10.0.2.182 (10.0.2.182) 56(84) bytes of data.
64 bytes from 10.0.2.182: icmp_seq=1 ttl=64 time=2.06 ms
64 bytes from 10.0.2.182: icmp_seq=2 ttl=64 time=1.08 ms
64 bytes from 10.0.2.182: icmp_seq=3 ttl=64 time=0.686 ms
^C
--- 10.0.2.182 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.686/1.277/2.064/0.579 ms
root@s2:~# ssh 10.0.2.182
root@10.0.2.182's password:
Permission denied, please try again.
root@10.0.2.182's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-71-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Jan 17 18:40:14 UTC 2026

System load:  0.103515625      Processes:           92
Usage of /:   75.3% of 2.73GB   Users logged in:    1
Memory usage: 52%             IPv4 address for ens3: 10.0.2.182
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.
   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.
```

Figura 5: Verificación de acceso a la BBDD desde un servidor

3.5. Acceso al servidor de administración desde el exterior

Esta sección demuestra que el acceso desde el exterior se limita al servidor de administración y, además, que se encuentra restringido al puerto SSH configurado.

En primer lugar, se intentó comprobar la conectividad mediante ping a la IP flotante del servidor admin. La prueba falla, lo cual es consistente con una configuración donde el tráfico ICMP desde el exterior no está permitido.

```
root@terraform:~/shared# ping 10.0.10.108
PING 10.0.10.108 (10.0.10.108) 56(84) bytes of data.
^C
--- 10.0.10.108 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2050ms
```

Figura 6: Comprobación de la conectividad al admin desde el exterior

A continuación, se realizó el acceso mediante SSH utilizando el puerto 2025, verificando que el acceso sí es posible cuando se usa el servicio y puerto permitidos.

```

root@terraform:~/shared# ssh -p 2025 root@10.0.10.108
The authenticity of host '[10.0.10.108]:2025 ([10.0.10.108]:2025)' can't be established.
ED25519 key fingerprint is SHA256:5Rj3uo9zWC3mrHooESNQiiPnATVf/VafPf9Yqs2+YBU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.0.10.108]:2025' (ED25519) to the list of known hosts.
root@10.0.10.108's password:
root@admin:~# █

```

Figura 7: Conexión a admin mediante ssh

Finalmente, se comprueba adicionalmente que desde el exterior no existe acceso directo al servidor de base de datos, reforzando el objetivo de aislamiento del componente de datos.

```

root@terraform:~/shared# ping 10.0.2.182
PING 10.0.2.182 (10.0.2.182) 56(84) bytes of data.
^C
--- 10.0.2.182 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3054ms

```

Figura 8: Intento de conexión a la base de datos

3.6. Acceso a los servidores web desde el admin

En este apartado se evidencia que los servidores web no están concebidos para ser accesibles directamente desde el exterior, sino desde el plano de administración interno. Para ello se realizaron pruebas de conectividad hacia s1, s2 y s3:

- Se muestra primero un intento de acceso directo (no permitido).
- A continuación, se muestra la conectividad correcta al realizar las pruebas desde el servidor admin, que sí está conectado a las redes internas y actúa como punto de gestión.

```

root@terraform:~/shared# ping 10.0.1.154
PING 10.0.1.154 (10.0.1.154) 56(84) bytes of data.
^C
--- 10.0.1.154 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2036ms

root@terraform:~/shared# ssh -p 2025 root@10.0.10.108
root@10.0.10.108's password:
Last login: Sat Jan 17 18:44:50 2026 from 10.0.10.12
root@admin:~# ping 10.0.1.154
PING 10.0.1.154 (10.0.1.154) 56(84) bytes of data.
64 bytes from 10.0.1.154: icmp_seq=1 ttl=64 time=0.745 ms
64 bytes from 10.0.1.154: icmp_seq=2 ttl=64 time=1.02 ms
64 bytes from 10.0.1.154: icmp_seq=3 ttl=64 time=0.779 ms
^C
--- 10.0.1.154 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2027ms
rtt min/avg/max/mdev = 0.745/0.846/1.015/0.120 ms

```

Figura 9: Conexión a servidor web desde admin

3.7. Acceso desde el interior a cualquier destino

Para verificar la conectividad general desde las instancias internas, se realizaron pruebas de acceso a destinos externos mediante ping. En la figura correspondiente se observa la conectividad desde un servidor interno hacia 8.8.8.8, validando que existe salida hacia el exterior para los nodos que deben disponer de ella.

```
root@s1:~# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=110 time=7.32 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=110 time=3.16 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=110 time=2.95 ms
^C
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 2.950/4.476/7.321/2.013 ms
```

Figura 10: Conectividad a internet desde el servidor web

De igual manera, se verifica la conectividad a Internet desde el servidor de administración.

```
root@admin:~# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=110 time=3.58 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=110 time=2.48 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=110 time=2.35 ms
^C
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 2.352/2.805/3.581/0.551 ms
```

Figura 11: Conectividad a internet desde admin

Por último, se comprueba que el servidor de base de datos no dispone de conectividad a Internet, coherente con el diseño de aislamiento hacia el exterior.

```
root@bbdd:~# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2036ms
```

Figura 12: Aislamiento de la base de datos